

Agenda

Zielgruppe: DevOps-Teams in regulierten Branchen (Versicherung, Banken) **Teilnehmer:** 2 Personen — vollständig Hands-on **Umgebung:** GitLab.com + eigener Docker Runner auf Server/VM **Leitfrage:** Wie betreibe ich Gemini CLI sicher, wie messe ich ob es besser wird, und wie stelle ich sicher, dass Guardrails nach jeder Änderung noch halten?

Roter Faden

Tag 1: Verstehen – Was ist im Image? Was tut der Agent wirklich?
Basis-Metriken erfassen (Ausgangszustand dokumentieren)

Tag 2: Automatisiert verbessern – GitLab Pipeline als Verbesserungsmotor
Jeder MR = ein Verbesserungsschritt mit messbarem Ergebnis
Guardrails definieren, testen, in Pipeline integrieren

Beide Tage arbeiten mit demselben Objekt: dem offiziellen Gemini CLI Image. Am Ende haben die Teilnehmer ein gehärtetes Image + Pipeline + DORA-Nachweis.

Tag 1: Verstehen, messen, Baseline setzen

09:00 — Einstieg (30 min)

- Warum KI-Tools in regulierten Umgebungen besondere Sorgfalt brauchen
- DORA: Was verlangt der Gesetzgeber konkret — und was nicht
- Die drei Kernfragen des Workshops:
 1. Was tut Gemini CLI wirklich (Image, Prozesse, Netzwerk)?
 2. Wie messe ich ob es sicher genug ist — und ob es besser wird?
 3. Wie stelle ich sicher, dass Guardrails nach einer Änderung noch halten?

09:30 — Gemini CLI lokal verstehen & nutzen (75 min)

Theorie (20 min)

- Gemini CLI Architektur: CLI → Tool-Aufrufe → Sandbox
- Sandboxing-Konzept: Was wird isoliert, was nicht?
- Sandbox-Modi im Vergleich:

Modus	Isolation	Wann sinnvoll
Kein Sandbox	Keine	Nur Entwicklung, nie Produktion
Docker Sandbox	Container	Standard für Teams
Docker + MicroVM	Kernel-Ebene	Hochsicherheit, Banken

Praxis (55 min) — beide TN parallel

```
# Schritt 1: Image holen und grundlegend verstehen
docker pull ghcr.io/google/gemini-cli:latest
docker inspect ghcr.io/google/gemini-cli:latest | jq '[0].Config'
docker history ghcr.io/google/gemini-cli:latest
```

```
# Schritt 2: Was läuft tatsächlich?
docker run --rm ghcr.io/google/gemini-cli:latest whoami
docker run --rm ghcr.io/google/gemini-cli:latest id
docker run --rm ghcr.io/google/gemini-cli:latest cat /etc/os-release

# Schritt 3: Gemini CLI mit Sandbox lokal nutzen
gemini --sandbox docker "Lies die Datei ./README.md und erkläre sie"

# Schritt 4: Sandbox-Grenzen beobachten
# Was passiert wenn der Agent auf etwas zugreift das er nicht soll?
gemini --sandbox docker "Führe curl google.com aus"
```

Diskussion: Was habt ihr beobachtet? Was hat die Sandbox geblockt, was nicht?

10:45 — Kaffeepause (15 min)

11:00 — Baseline messen: Der Ausgangszustand dokumentieren (90 min)

Das ist der wichtigste Block von Tag 1 — ohne Baseline keine Aussage ob sich etwas verbessert hat.

Theorie (25 min)

Drei Messdimensionen:

Dimension	Tool	Was wird gemessen
Schwachstellen (CVEs)	Trivy, Gripe	Anzahl Critical/High/Medium, Risk Score
CIS Compliance	Trivy --compliance	Pass/Fail-Rate gegen CIS Docker Benchmark
Guardrails	Eigene Tests	Was darf der Agent? Was ist geblockt?

Warum Gripe zusätzlich zu Trivy?

- Trivy: breiter Scope (CVEs + Config + Secrets)
- Gripe: Risk Score 0-10 kombiniert CVSS + EPSS (Exploit-Wahrscheinlichkeit) + KEV
- Beide zusammen geben ein vollständigeres Bild

Praxis (65 min) — Baseline-Tabelle füllen

```
# CVE-Scan mit Trivy
trivy image --format json \
  ghcr.io/google/gemini-cli:latest > baseline-trivy.json

# Metriken extrahieren
cat baseline-trivy.json | jq '{
  critical: [.Results[].Vulnerabilities[]? | select(.Severity=="CRITICAL")] | length,
  high:     [.Results[].Vulnerabilities[]? | select(.Severity=="HIGH")] | length,
  medium:   [.Results[].Vulnerabilities[]? | select(.Severity=="MEDIUM")] | length,
  packages: [.Results[].Packages[]?] | length
}'

# CIS Docker Benchmark
```

```
trivy image --compliance docker-cis \
  ghcr.io/google/gemini-cli:latest > baseline-cis.txt

# CIS Pass-Rate berechnen
grep -c "PASS" baseline-cis.txt
grep -c "FAIL" baseline-cis.txt

# Risk Score mit Grype (kombiniert CVSS + EPSS + KEV)
grype ghcr.io/google/gemini-cli:latest --output json > baseline-grype.json

# SBOM generieren (DORA Art. 19 – Third-Party Risk Dokumentation)
trivy image --format cyclonedx \
  --output baseline-sbom.json \
  ghcr.io/google/gemini-cli:latest
```

Jeder TN füllt seine Baseline-Tabelle:

Metrik	Ausgangswert	Zielwert	Nach MR #1	Nach MR #2	...
CVE Critical	?	0			
CVE High	?	<5			
CIS Pass-Rate	?%	>85%			
Grype Risk Score	?	<3.0			
Package-Anzahl	?	minimieren			
Image-Größe (MB)	?	minimieren			

Ergebnis: Jeder TN hat seinen persönlichen Ausgangszustand dokumentiert.

13:00 — Mittagspause (60 min)

14:00 — Guardrails: Was darf der Agent — und wie teste ich das? (105 min)

Theorie (35 min)

Was sind Guardrails bei Gemini CLI?

```
Ebene 1: Sandbox (Docker/MicroVM)
└─ Was darf der Container? Netzwerk, Dateisystem, Syscalls

Ebene 2: Tool-Permissions (settings.json / GEMINI.md)
└─ Welche Tools darf der Agent aufrufen?

Ebene 3: Behavior (Was tut der Agent mit erlaubten Tools?)
└─ Macht er das Erwartete, oder etwas Unerwartetes?
```

Wichtig: Ebene 3 (GEMINI.md) ist kein technischer Guardrail GEMINI.md sind Instruktionen an das LLM — kein Hard Block. Ein Prompt-Injection-Angriff kann diese Ebene umgehen. Nur Ebene 1 und 2 sind technisch durchsetzbar.

Allowlist vs. Blocklist — was gilt für DORA?

Ansatz	Sicherheit	DORA-Anforderung
Allowlist (tools.core)	Stärker — alles verboten was nicht explizit erlaubt	Keine Pflicht, aber empfohlen
Blocklist (tools.exclude)	Schwächer — nur bekannte Angriffe geblockt	Zulässig mit Dokumentation

DORA schreibt keinen der beiden Ansätze vor. DORA verlangt: die Entscheidung dokumentieren und begründen. Wer Blocklist nutzt, muss erklären warum das fuer das eigene Risikoprofil ausreicht.

Das Testprinzip:

Positive Tests:	Agent SOLL etwas können → muss funktionieren
Negative Tests:	Agent SOLL etwas NICHT können → muss geblockt sein
Regression:	Nach Update → beide Tests nochmal → Ergebnis gleich?

Praxis (70 min)

```
# Test 1: Netzwerk-Guardrail (Negativ-Test)
# Agent SOLL kein Netzwerk haben
docker run --rm \
  --network none \
  ghcr.io/google/gemini-cli:latest \
  sh -c "curl https://example.com 2>&1 | head -1"
# Erwartetes Ergebnis: "curl: (6) Could not resolve host"

# Test 2: Dateisystem-Guardrail (Negativ-Test)
# Agent SOLL nicht schreiben können
docker run --rm \
  --read-only \
  ghcr.io/google/gemini-cli:latest \
  sh -c "touch /tmp/test 2>&1"
# Erwartetes Ergebnis: "Read-only file system"

# Test 3: Privilege Escalation (Negativ-Test)
docker run --rm \
  --security-opt no-new-privileges \
  ghcr.io/google/gemini-cli:latest \
  sh -c "sudo id 2>&1"
# Erwartetes Ergebnis: Fehler — kein sudo

# Test 4: Positiv-Test — Agent SOLL eine Datei lesen können
docker run --rm \
  --read-only \
  --network none \
  -v $(pwd)/tests/input.txt:/workspace/input.txt:ro \
  ghcr.io/google/gemini-cli:latest \
  sh -c "cat /workspace/input.txt"
# Erwartetes Ergebnis: Dateiinhalt erscheint

# Test 5: Behavioral Test — Output-Vergleich (Golden Test)
docker run --rm \
  --network none \
```

```
ghcr.io/google/gemini-cli:latest \
gemini run my-skill < tests/input.txt > actual.txt
diff tests/expected.txt actual.txt
# Kein Output = Test bestanden
```

Guardrail-Scorecard (analog zur CVE-Tabelle):

Guardrail	Soll-Verhalten	Aktuell	Nach Härtung
Netzwerk	Geblockt	?	
Filesystem-Write	Geblockt	?	
Privilege Escalation	Geblockt	?	
Skill-Output stabil	Gleicher Output	?	
Unbekannte Tool-Calls	Geblockt	?	

Diskussion: Was war überraschend? Wo halten die Guardrails nicht?

15:45 — Kaffeepause (15 min)

16:00 — Tag 1 Zusammenfassung: Ausgangslage ist klar (45 min)

- Jeder TN präsentiert seine Baseline-Tabelle (10 min je TN)
 - Was sind die größten Lücken? Wo liegt der Fokus für Tag 2?
 - Reihenfolge der MRs festlegen: was hat den größten Impact?
 - Fragen und offene Punkte sammeln
-

Tag 2: Automatisiert verbessern — die Pipeline als Verbesserungsmotor

09:00 — Einstieg Tag 2 (15 min)

- Recap: Baseline-Werte, geplante MRs
 - Ziel heute: Pipeline baut das bessere Image, jeder MR ist messbar
-

09:15 — GitLab Pipeline aufsetzen (60 min)

Theorie (15 min)

- Warum GitLab.com für Training, eigener Runner für Produktion (DORA Art. 9)
- Pipeline-Struktur: Stages als Qualitäts-Gates

Praxis (45 min) — beide TN registrieren ihren Runner

```
# Runner auf Server registrieren
gitlab-runner register \
  --url https://gitlab.com \
  --token $RUNNER_TOKEN \
  --executor docker \
  --docker-image docker:latest \
```

```
--non-interactive

# Runner sichern: privileged = false ist Pflicht
# /etc/gitlab-runner/config.toml
[[runners]]
  [runners.docker]
    privileged = false
    allowed_pull_policies = ["if-not-present"]
    disable_entrypoint_overwrite = true
```

Basis-Pipeline die alle TN als Startpunkt bekommen:

```
# .gitlab-ci.yml - Startpunkt (noch ohne Security)
stages:
  - scan
  - build
  - test

variables:
  IMAGE: "ghcr.io/google/gemini-cli:latest"

scan-baseline:
  stage: scan
  image: aquasec/trivy:latest
  script:
    - trivy image --format json --output trivy.json $IMAGE
    - cat trivy.json | jq '{
      critical: [.Results[].Vulnerabilities[]? | select(.Severity=="CRITICAL")] |
length,
      high: [.Results[].Vulnerabilities[]? | select(.Severity=="HIGH")] | length
    }'
  artifacts:
    paths: [trivy.json]
```

10:15 — Kaffeepause (15 min)

10:30 — MR-Schleife: Schrittweise verbessern (150 min)

Konzept: Messen vor Erzwingen

Die Pipeline-Gates starten als `WARN`, nicht als `FAIL`. Das ist Absicht.

Ein Gate das sofort als `FAIL` konfiguriert ist bevor man den Zielwert erreicht hat, blockiert jeden Commit von Anfang an — bevor eine einzige Verbesserung gemacht wurde. Ergebnis: alle Pipelines dauerhaft rot, niemand nimmt die Warnungen mehr ernst.

Der richtige Ablauf:

```
1. MESSEN      Pipeline laeuft durch, zeigt den Istzustand
               "CIS Pass-Rate: 55% - Ziel: 80%"
```

2. VERSTEHEN Warum ist der Wert schlecht? Welcher MR hilft am meisten?
3. VERBESSERN MR nach MR bis der Zielwert erreicht ist
"55% → 70% → 82%"
4. ERZWINGEN Gate einschalten: ab jetzt wird der Wert erzwungen
Ein kuenftiger MR der unter 80% faellt wird geblockt

Konkret in der Pipeline — TN kommentieren diese Zeile ein sobald ihr Zielwert stabil erreicht ist:

```
# Erst WARN:
echo "WARN: CIS Pass-Rate $RATE% unter Ziel 80%"
# exit 1  ← auskommentiert waehrend der Verbesserungsphase

# Spaeter FAIL (einkommentieren wenn Ziel erreicht):
# exit 1  ← ab jetzt blockiert die Pipeline jeden Rueckschritt
```

In echten Unternehmen ist das derselbe Fehler: Gates zu frueh zu streng gesetzt, dauerhaft rote Pipelines, Warnungen werden ignoriert.

Jeder MR hat dieselbe Struktur:

1. Pipeline findet Problem oder TN entscheidet nächsten Schritt
2. TN öffnet MR mit Dockerfile-Änderung
3. Pipeline läuft → Metriken werden verglichen
4. TN sieht: besser / schlechter / neutral → mergt oder korrigiert

MR #1: Non-root User

```
# Vorher (im offiziellen Image):
# USER root (implizit)

# Nachher:
RUN groupadd --gid 1000 gemini && \
    useradd --uid 1000 --gid gemini --shell /bin/sh gemini
USER 1000:1000
```

Pipeline-Ergebnis: CIS Check 1.1 Ensure a user is created → PASS

MR #2: Schlankeres Base Image (größter CVE-Impact)

```
# Vorher:
FROM node:20-slim

# Nachher:
FROM node:20-alpine
```

Pipeline-Ergebnis: CVE Critical von X auf 0, Packages von ~400 auf ~190

MR #3: Capabilities dropen

```
# Im docker-compose.yml / Deployment:
security_opt:
  - no-new-privileges:true
cap_drop:
  - ALL
cap_add:
  - NET_BIND_SERVICE    # nur wenn wirklich nötig
```

MR #4: Read-only Filesystem + tmpfs für Schreibzugriffe

```
read_only: true
tmpfs:
  - /tmp:size=100m,mode=1777
```

Pipeline-Metriken nach jedem MR automatisch in Kommentar:

```
# In .gitlab-ci.yml ergänzen:
metrics-comment:
  stage: scan
  script:
    - |
      CRITICAL=$(cat trivy.json | jq ' [.Results[] .Vulnerabilities[]? |
select(.Severity=="CRITICAL")] | length')
      HIGH=$(cat trivy.json | jq ' [.Results[] .Vulnerabilities[]? |
select(.Severity=="HIGH")] | length')
      echo "CVE Critical: $CRITICAL | High: $HIGH"
  artifacts:
    reports:
      dotenv: metrics.env
```

Metriken-Tabelle wird nach jedem MR aktualisiert — TN sehen live wie sich ihre Zahlen verbessern.

13:00 — Mittagspause (60 min)

14:00 — Guardrails in die Pipeline integrieren (75 min)

Theorie (15 min)

- Guardrail-Tests laufen bei jedem MR automatisch
- Negativ-Test schlägt fehl → Pipeline-Fehler → MR kann nicht gemergt werden
- Positiv-Test schlägt fehl → Regression → sofort sichtbar

```
# .gitlab-ci.yml - Guardrail-Test-Stage ergänzen
guardrail-tests:
  stage: test
  script:
    # settings.json validieren - Struktur und Entscheidung dokumentieren
    - |
      python3 -c "
```



```

import json, sys
cfg = json.load(open('settings.json'))
assert 'tools' in cfg, 'FAIL: tools section fehlt in settings.json'
if 'core' in cfg['tools']:
    print('INFO: Allowlist aktiv – stärkste Isolation')
elif 'exclude' in cfg['tools']:
    print('WARN: nur Blocklist aktiv – schwächer als Allowlist')
    print('      DORA: Begründung in docs/guardrail-decision.md
erforderlich')
else:
    print('WARN: keine Tool-Permissions konfiguriert')
    sys.exit(1)
"

# Negativ-Test: kein Netzwerk
- |
result=$(docker run --rm --network none $IMAGE \
    sh -c "curl https://example.com 2>&1" || true)
echo "$result" | grep -q "Could not resolve\|Network unreachable" \
    && echo "PASS: network blocked" \
    || (echo "FAIL: network NOT blocked" && exit 1)

# Negativ-Test: kein Schreiben
- |
docker run --rm --read-only $IMAGE \
    sh -c "touch /test 2>&1" \
    | grep -q "Read-only" \
    && echo "PASS: filesystem read-only" \
    || (echo "FAIL: filesystem writable" && exit 1)

# Behavioral Test: Output-Stabilität
- |
docker run --rm --network none $IMAGE \
    gemini run smoke-test < tests/input.txt > actual.txt
diff tests/expected.txt actual.txt \
    && echo "PASS: behavior unchanged" \
    || (echo "FAIL: behavior changed after update" && exit 1)

```

Praxis (60 min)

TN ergänzen ihre Pipeline um Guardrail-Tests und simulieren eine Regression:

```

# Simulation: was passiert wenn ein Update die Guardrails bricht?
# TN entfernt bewusst --read-only → Pipeline schlägt fehl → Pipeline fängt es auf

```

15:15 — Kaffeepause (15 min)

15:30 — DORA-Nachweis: Was die Pipeline automatisch produziert (60 min)

Theorie (20 min)

--	--	--

DORA-Artikel	Anforderung	Pipeline-Artefakt
Art. 8	Risiken identifizieren & dokumentieren	Trivy JSON-Report, Gripe Risk Score
Art. 9	Schutz & Prävention	CIS-Compliance-Report, Guardrail-Test-Logs
Art. 10	Erkennung	Pipeline schlägt bei neuen CVEs an
Art. 13	Testing-Nachweis	Guardrail-Tests, Behavioral Tests als Artefakte
Art. 19	Third-Party Risk	SBOM (CycloneDX-Format)

Praxis (40 min)

```
# Vollständige DORA-Artefakt-Stage
dora-evidence:
  stage: scan
  script:
    # SBOM (Art. 19)
    - trivy image --format cyclonedx --output sbom.json $IMAGE

    # CIS-Compliance-Report (Art. 9)
    - trivy image --compliance docker-cis --format template
      --template "@contrib/html.tpl" --output cis-report.html $IMAGE

    # Vulnerability-Report (Art. 8)
    - trivy image --format template
      --template "@contrib/html.tpl" --output vuln-report.html $IMAGE
  artifacts:
    paths:
      - sbom.json
      - cis-report.html
      - vuln-report.html
    expire_in: 1 year # Audit-Aufbewahrungsfrist
```

MR-History in GitLab = automatisches Änderungsprotokoll = DORA Art. 8 Nachweis

16:30 — Abschluss: Was habt ihr gebaut? (45 min)

Jeder TN präsentiert (15 min je TN):

- Vorher/Nachher-Tabelle: CVEs, CIS-Score, Guardrail-Status
- Welcher MR hatte den größten Impact?
- Welche Guardrail war überraschend?

Das Ergebnis:

Ausgangszustand:	Endstand:
CVE Critical: 5 →	CVE Critical: 0
CIS Pass-Rate: 60% →	CIS Pass-Rate: 88%
Guardrails: manuell →	Guardrails: automatisch getestet
DORA-Nachweis: keiner →	SBOM + Reports + MR-History

Nächste Schritte für die eigene Umgebung (20 min)

1. Eigenen Runner auf internem Server → Daten verlassen nicht das Rechenzentrum
 2. CIS-Zielwert als Pipeline-Gate definieren (`exit 1` unter 80%)
 3. SBOM-Retention-Policy in GitLab setzen (1 Jahr für DORA-Audit)
 4. Guardrail-Tests für eigene Skills ergänzen
-

Vorbereitungsliste (Trainer)

Vor dem Workshop:

- ☐ Trivy-Scan des offiziellen Images vorab durchführen — Findings kennen
- ☐ `tests/input.txt` und `tests/expected.txt` vorbereiten (Behavioral Tests)
- ☐ `.gitlab-ci.yml` -Startpunkt als Template im GitLab-Projekt hinterlegen
- ☐ Server/VM mit Docker für jeden TN vorbereitet (Runner-Token bereit)
- ☐ Gripe installiert (`curl -sSfL`
`https://raw.githubusercontent.com/anchore/grype/main/install.sh | sh`)

Materialien:

- `tests/` — Input + Expected Output für Behavioral Tests
- `.gitlab-ci.yml.template` — Startpunkt für TN (ohne Security, zum Ausbauen)
- `docs/dora-mapping.md` — DORA-Artikel zu Pipeline-Schritt-Mapping
- `docs/baseline-template.md` — Leere Tabelle zum Ausfüllen (Tag 1)

Materialien

- `tests/` — Behavioral Test Inputs und Expected Outputs
- `policies/` — OPA/Conftest Policies (optional, für Erweiterungen)
- `.gitlab-ci.yml.template` — Pipeline-Startpunkt
- `docs/dora-mapping.md` — DORA-Artikel Mapping

Was tut Gemini CLI wirklich (Image, Prozesse, Netzwerk)?